Reliable Transfer for a Disaster-Response Network: Go-Back-N vs Selective Repeat

CS3001 Computer Networks, Assignment-II · Muhammad Hammad(24k0544) · Random seed: 544

1. Approach

The EOC and the field teams are connected by a satellite link with R = 2 Mbps, RTT = 500 ms and L = 8,000 bits per packet. The link loses, corrupts and delays both packets and ACKs, so the transport protocol must deliver every packet correctly and in order. The design started from the link numbers. The transmission time is L/R = 4 ms, so with Stop-and-Wait the utilization is U = 4 / (500 + 4) ≈ 0.79%. The link would be idle more than 99% of the time. Pipelining with a window of N = 4 raises this to 16/504 ≈ 3.17% (about 15.9 kbps to 63.5 kbps), so a sliding-window protocol is needed.

Both sliding-window protocols were built and compared. Go-Back-N (GBN) is simpler, with one timer and a receiver that keeps only `expectedseqnum`. Selective Repeat (SR) needs per-packet timers and a receiver buffer, but it retransmits only what is missing. Because every retransmission costs a lot on this long-delay link, SR was expected to be the better fit, and the experiments test that expectation. All testing uses a deterministic discrete-event simulator seeded with the last four digits of the roll number, so every run can be reproduced exactly.

2. Implementation

The code is plain Python with no transport library. `task3.py` contains six parts:

- **Packet and checksum.** A packet carries a type (DATA or ACK), a sequence number, a payload and a 16-bit one's-complement checksum over all fields. Corruption flips one random bit, and the receiver recomputes the checksum. A mismatch is treated as a loss.
- **Event simulator.** A heap of timed events drives everything. A timer is an event that can be cancelled, so there are no threads and runs are deterministic.
- **Unreliable channel.** Each direction has a transmitter with transmission time L/R and 250 ms propagation delay. Each packet and ACK is independently

lost, corrupted or delayed (an extra 0.05–0.2 s) with the configured probabilities.

- **GBN.** The sender keeps `base`, `nextseqnum` and a single timer for the oldest unACKed packet. The receiver delivers only the packet whose number equals `expectedseqnum`, discards anything else and re-sends the cumulative ACK of the last in-order packet. On timeout, the sender resends every packet from `base` to `nextseqnum-1`.
- **SR.** The sender keeps an acked flag and a timer for each packet, and the window slides past contiguous acked packets. The receiver has a window starting at `rcv_base`. An in-window packet is buffered and individually ACKed, and buffered packets are delivered in order. A packet from the previous window is ACKed again, because its ACK was probably lost. On timeout, only that packet is resent.
- **Runner.** The command line sets the protocol, number of packets, window N, sequence bits k, loss, corruption and delay probabilities, timeout and seed. Every send, receive, ACK, loss, corruption, timeout and retransmission produces a time-stamped log line.

Internally both protocols use absolute counters, and the wire carries `n mod 2^k`. Window membership is tested with `(seq − base) mod 2^k < N`, so wrap-around works (200 packets with k = 3 wrap 25 times). The program refuses windows above the safe limit (GBN: $N \leq 2^k - 1$; SR: $N \leq 2^{(k-1)}$) unless `--allow-unsafe` is given. After each run the delivered data is compared with the data sent, and a `correct` flag is reported. Over 600 runs (both protocols, three scenarios, 100 seeds) the delivered data was correct every time.

3. Experiments

Both protocols were run with 200 packets, N = 4 and k = 3 under three scenarios: **A** reliable channel; **B** packet loss 0.10; **C** mixed errors (loss 0.05, corruption 0.05, delay 0.10). The metrics are packets transmitted (including retransmissions), retransmissions, packets delivered, execution time and throughput (delivered bits divided by time). The retransmission timeout is 1.0 s and the maximum extra delay is 0.2 s.

The timeout choice matters. Early runs with a 0.6 s timeout delivered wrong data in scenario C. The worst-case RTT with delay is about 0.95 s, so a short timeout caused spurious retransmissions. Stale copies of those packets survived long

enough to match a reused 3-bit sequence number. Loss and corruption alone were safe with any timeout. A timeout above the worst-case RTT removed the problem.

4. Results

Table 1. Single run, seed 544 (200 packets, N = 4, k = 3).

Scenario	Protocol	Transmitted	Retransmissions	Delivered	Time (s)	Throughput (kbps)
A: Reliable	GBN	200	0	200	25.22	63.44
A: Reliable	SR	200	0	200	25.22	63.44
B: Loss 0.10	GBN	327	127	200	63.45	25.22
B: Loss 0.10	SR	253	53	200	55.77	28.69
C: Mixed	GBN	384	184	200	87.55	18.27
C: Mixed	SR	246	46	200	54.15	29.55

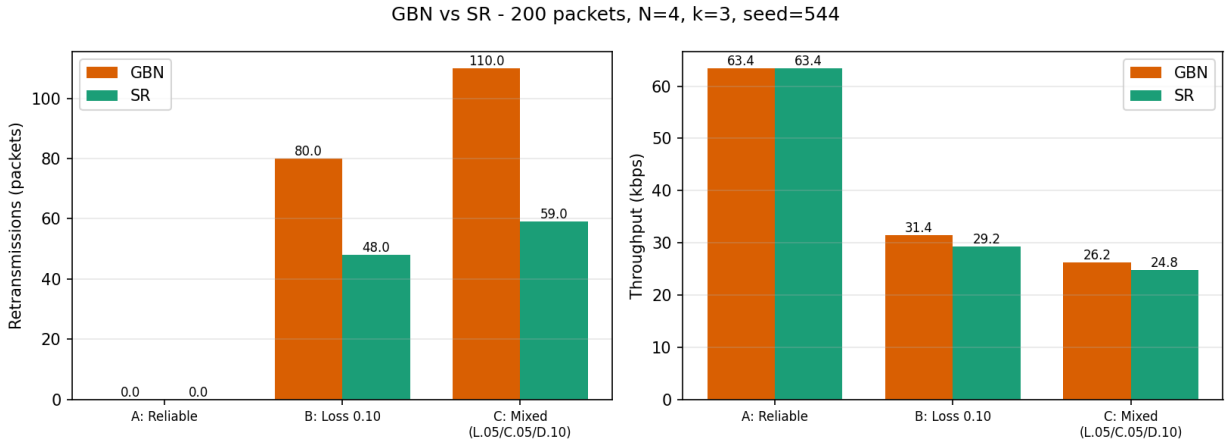
Table 2. Average of 20 runs (consecutive seeds).

Scenario	Protocol	Transmitted	Retransmissions	Time (s)	Throughput (kbps)
B: Loss 0.10	GBN	292.1	92.0	53.98	30.12
B: Loss 0.10	SR	247.3	47.3	54.57	29.51
C: Mixed	GBN	354.2	154.2	76.79	21.37
C: Mixed	SR	246.3	46.4	55.32	29.02

In scenario A the throughput of 63.44 kbps matches the analytical value of about 63.5 kbps for N = 4, which confirms the simulator timing. All 200 packets were delivered in every run.

5. Comparison

On a reliable channel the two protocols behave identically, because neither ever needs to recover. Once packets are lost, GBN retransmits far more than SR (127 against 53 under loss, 184 against 46 under mixed errors), since its receiver discards every packet after the missing one and its sender then resends the whole outstanding window. SR buffers out-of-order packets, ACKs each one individually and resends only the missing packet. Delay hurts GBN most in scenario C, because delayed packets arrive out of order and are thrown away, which forces it to wait for timeouts. SR accepts them into its buffer, so its throughput stays near 29 kbps while GBN falls to 18–21 kbps. The 20-run average for scenario B shows that the two finish in almost the same time (about 54 s). The link is only about 3% utilised, so GBN's extra copies fit into idle capacity and cost bandwidth rather than time. SR's advantage under loss is therefore mainly fewer transmitted packets, and it becomes a clear time advantage once delay and reordering are present.



6. Conclusion

SR is the more suitable protocol for this network. It uses less of a limited-bandwidth link, since it sends 21–35% fewer packets than GBN under errors, and it handles delayed and reordered packets without extra timeouts. The price is more state: per-packet timers and a receive buffer. Two safety conditions apply: the window must not exceed 2^(k−1) for SR (2^k − 1 for GBN), and the timeout must exceed the worst-case RTT, otherwise stale duplicates can be mistaken for new packets. With these conditions, both implementations delivered all data correctly and in order in every test run.

Demo Video Link:
https://drive.google.com/drive/folders/1bfIT2tfCdfKoO7I_apvWzPpVAIqQnYNN?usp=sharing